Multiple Choice Answers (1-8, 11-12)

1. **B** – The learning rate is too high
2. **B** – Reducing variance at the cost of a small increase in bias
3. **C** – Random Forests always have lower bias than a single fully-grown decision tree
4. **B** – Narrows the margin and penalizes misclassifications more heavily
5. **B** – Fits a linear regression on training data and predicts target values for test data
6. **B** – Performs K-Means clustering with 3 clusters and stores the cluster assignment for each point
7. **C** – The forest will contain 100 decision trees
8. **C** – from sklearn.tree import DecisionTreeClassifier; model = DecisionTreeClassifier()

Question No. 9: Simple Linear Regression:

$x = [1, 2, 3, 4]$ $y = [2, 2, 4, 5]$

a) Means

$$\bar{x} = \frac{1+2+3+4}{4} = \frac{10}{4} = 2.5$$

$$\bar{y} = \frac{2+2+4+5}{4} = \frac{13}{4} = 3.25$$

b) Compute w and b

$$w = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	Product	square
1	2	$1-2.5=-1.5$	-1.25	1.875	2.25
2	2	-0.5	-1.25	0.625	0.25
3	4	0.5	0.75	0.375	0.25
4	5	1.5	1.75	2.625	2.25

sum of products $= 1.875 + 0.625 + 0.375 + 2.625$
$= 5.5$

Sum of squares $= 2.25 + 0.25 + 0.25 + 2.25$
$= 5.0$

$w = 5.5 / 5.0 = 1.1$

$b = \bar{y} - w\bar{x} = 3.25 - (1.1)(2.25) = 3.25 - 2.75 = 0.5$

c) Predicted for $x = 5$

$\hat{y} = 1.1(5) + 0.5 = 5.5 + 0.5 = 6.0$

Residual for (3, 4)

Predicted at $x = 3$: $1.1(3) + 0.5 = 3.3 + 0.5 = 3.8$

Residual $= 4 - 3.8 = 0.2$

Question 10 - Binary Classifier Performance

$P(y=1|x) = [0.9, 0.4, 0.8, 0.3, 0.6]$

$y_true = [1, 0, 0, 1, 1]$ Threshold = 0.5

a) Metrics:

- $0.9 \rightarrow 1$
- $0.4 \rightarrow 0$
- $0.8 \rightarrow 1$
- $0.3 \rightarrow 0$
- $0.6 \rightarrow 1$

So, prediction = $[1, 0, 1, 0, 1]$

Confusion Matrix (class 1 = positive):

	Pred 1	Pred 0
True 1	2(TP)	1(FN)
True 0	1(FP)	1(TN)

$$\text{Accuracy} = \frac{TP+TN}{total} = \frac{2+1}{5} = \frac{3}{5} = 0.6$$

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{2}{2+1} = \frac{2}{3} = 0.667$$

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{2}{3} = 0.667$$

$$\text{F1-score} = 2 \cdot \frac{0.667 \cdot 0.667}{0.667 + 0.667} = 0.667$$

b) Scenario for threshold 0.3:

Threshold 0.3 → more points predicted as class 1 (higher recall, lower precision).

Example: Disease screening - we want to check most sick people (minimize false negatives), even if we get more false positives. Missing a disease (FN) is far worse than a false alarm (FP).

Question 11 - Decision Tree Gini Impurity

Given Data:

- Parent node: 6 Positives (P), 4 Negatives (N) → total = 10
- Left child: 5P, 1N → total = 6
- Right child: 1P, 3N → total = 4

a) Gini Impurity of the parent node:

Formula: $Gini = 1 - (P_p^2 + P_n^2)$

where P_p = proportion of positives, P_n = proportion of negatives.

$P_p = 6/10 = 0.6$, $P_n = 4/10 = 0.4$

$Gini = 1 - (0.6^2 + 0.4^2) = 1 - (0.36 + 0.16) = 1 - 0.52 = \boxed{0.48}$

b) Weighted Gini Impurity of the children:

Left child Gini:

$P_p = 5/6 = 0.833$, $P_n = 1/6 = 0.1667$

$Gini_{left} = 1 - (0.8333^2 + 0.1667^2) = 1 - (0.6944 + 0.0278)$

$= 1 - 0.7222 = 0.2778$

Right child Gini:

$P_p = 1/4 = 0.25$, $P_n = 3/4 = 0.75$

$Gini_{right} = 1 - (0.25^2 + 0.75^2) = 1 - (0.0625 + 0.5625) = 1 - 0.625$

$= 0.375$

Weighted Average:

$Gini_{children} = 6/10 \times 0.2778 + 4/10 \times 0.375 = \boxed{0.3167}$

c) Gini gain and evaluation:

$Gain = Gini_{parent} - Gini_{children} = 0.48 - 0.3167 = 0.1633$

Is this a good split?

Yes _ Gini gain is positive (0.1633), meaning the split reduces impurity. The left child is mostly positive (5/6), the right child is mostly negative (3/4), so the split creates purer groups than the parent.

Question 12 – Fraud Detection & Misleading Accuracy

(a) Why is accuracy misleading? In fraud detection, cases of fraud usually make up a tiny fraction of the data (e.g., 1%). If a model simply predicts "Not Fraud" for every single transaction, it will be 99% accurate but completely fail to catch any actual fraud. Accuracy fails here because it treats all classes as equally important, ignoring the fact that missing a fraud case (the minority class) is the most critical error.

(b) Two concrete remedies

1. **Use Better Metrics:** Instead of accuracy, use Precision, Recall, or the F1-Score. Recall is especially important here because it measures what percentage of actual fraud cases the model successfully caught.
2. **Resampling or Class Weights:** You can use SMOTE (Oversampling) to create more examples of the fraud class during training, or apply "Class Weights" to the model so that it is penalized more heavily for missing a fraud case than for a false alarm.